Intelligenza Artificiale

Michele Castenedoli

1 ottobre 2026

Indice

1 Introduzione all'Intelligenza Artificiale **2**
1.1 Definizione e Storia 2
1.2 Tipi di Intelligenza Artificiale 2

2 Intelligent Agents **6**
2.1 Definizione 6
2.2 Razionalità 8
2.3 PEAS 10
2.4 Tipi di Ambiente 11
2.5 Tipi di Problemi 12
2.6 Programma Agente 12

3 Risolvere Problemi con la Ricerca **17**
3.1 Problem Solving Agents 17
3.2 Come Formulare un Problema 18
3.3 Tree-Search Algorithm 20
3.4 Ricerca Non Informata 24
3.5 Ricerca Informata 34

4 Esercizi **43**

Capitolo 1

Introduzione all'Intelligenza Artificiale

1.1 Definizione e Storia

Nel 1950 Alan Turing pubblica un articolo intitolato "Computing Machinery and Intelligence" in cui viene proposto un esperimento per determinare se una macchina può essere considerata dotata di intelligenza. Tale esperimento, noto come *Test di Turing*, coinvolge un interrogatore umano che comunica con due entità nascoste: una **macchina** e un **essere umano**. L'interrogatore sottopone domande a entrambe le entità e, basandosi sulle risposte fornite, deve determinare quale delle due è la macchina. Se l'interrogatore non è in grado di distinguere le due parti, la macchina è considerata intelligente.

In seguito l'attenzione si è spostata sulla ricerca di metodi utili per risolvere problemi che richiedono l'intelligenza umana mediante l'utilizzo di algoritmi e modelli matematici fino ad arrivare all'introduzione di reti neurali e deep learning.

> **Intelligenza Artificiale**
>
> L'**Intelligenza Artificiale** è la disciplina che si occupa di studiare come **simulare** l'intelligenza umana in scenari complessi.

1.2 Tipi di Intelligenza Artificiale

1.2.1 Autonomous Agents

Gli **Autonomous Agents** sono sistemi che percepiscono l'ambiente e agiscono autonomamente per raggiungere determinati obiettivi.

1.2.2 Data Analysis

Consiste nell'utilizzo di algoritmi per analizzare grandi quantità di dati ed estrarre informazioni utili e possibili correlazioni complesse.

1.2.3 Machine Learning

È lo sviluppo di algoritmi che permettono a modelli di apprendere da *dati di esempio* e migliorare le loro prestazioni nel tempo senza essere esplicitamente programmati: a differenza della programmazione tradizionale, in cui lo sviluppatore scrive le istruzioni, si fornisce soltanto un algoritmo di base e una grande quantità di dati, da cui il sistema ricava pattern, correlazioni e regole statistiche necessarie per svolgere il compito.

Il Machine Learning è suddiviso in tre categorie principali:

- **Unsupervised Learning**: il modello viene addestrato nel corso del tempo su un insieme di dati non etichettati (senza alcun "tag", categoria, o classificazione assegnati a priori da un essere umano). L'obiettivo è scoprire strutture nascoste o pattern nei dati, senza risposte corrette predefinite.

- **Supervised Learning**: il modello viene addestrato su un insieme di dati etichettati, dove ogni esempio di input è associato a una risposta corretta. L'obiettivo è che il modello impari a mappare gli input alle risposte corrette.

- **Reinforcement Learning**: il modello impara attraverso continue interazioni con l'ambiente, ricevendo ricompense o penalità a seconda delle azioni intraprese. L'obiettivo è massimizzare la ricompensa totale nel tempo. Pertanto il sistema valuta le conseguenze delle scelte a lungo termine anziché cercare un vantaggio immediato.

  Esempio: il sistema che simula un giocatore di scacchi potrebbe sacrificare la regina se prevede che in due mosse sarà in grado di dare scacco matto, raggiungendo così la massima ricompensa disponibile.

1.2.4 Time Series Analysis

La Time Series Analysis è un'area del Machine Learning che si concentra sull'analisi di dati raccolti nel tempo. Le time series sono sequenze di dati misurati a intervalli regolari (temperatura giornaliera, prezzi delle azioni o dati di vendita mensili). L'obiettivo è identificare pattern, tendenze, stagionalità[1] per fare previsioni future.

[1] Stagionalità: la presenza di fluttuazioni, variazioni o pattern prevedibili che si ripetono a intervalli regolari di tempo.

Gli approcci comuni per la Time Series Analysis sono:

- **Riconoscimento di anomalie e cause**: processo di identificazione di dati o eventi che si discostano significativamente dal comportamento normale o atteso. Queste anomalie possono fornire indicatori di problemi, errori o situazioni non previste che richiedono attenzione.
- **Generative Transformers**: sono una classe di modelli che permettono di predire il prossimo elemento di una sequenza di dati partendo dagli elementi precedenti (token di un testo, pixel di un'immagine). Si introduce il concetto di **attenzione**: meccanismo usato per pesare l'importanza delle diverse parti della sequenza in input durante la generazione di output.

  Esempio: Se il modello di apprendimento sta elaborando la sequenza di token relativa alla frase "ho mangiato una pesca", il meccanismo di attenzione assegnerà un peso (punteggio) molto elevato alle parole "mangiato" e "una" per aiutare il sistema a capire che ci si sta riferendo al frutto e non all'hobby di pescare.

1.2.5 Intelligent Agents

Un Intelligent Agent è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce sull'ambiente per raggiungere un obiettivo specifico e svolgere un determinato compito. Gli elementi cardine di un Intelligent Agent includono:

- **Performance Measure**: misurazione del successo (efficacia) dell'agente nel raggiungere i suoi obiettivi.
- **Rationality**: l'agente deve agire in modo da massimizzare la sua performance measure attesa, dati la sequenza percettiva e la conoscenza a priori di cui dispone.

Differenza – Intelligent Agents vs Autonomous Agents

I due concetti si sovrappongono in gran parte, ma pongono l'accento su aspetti diversi. L'**intelligenza** riguarda la *qualità delle decisioni*: un agente è intelligente (razionale) se sceglie le azioni che massimizzano la performance measure attesa. L'**autonomia** riguarda invece *quanto l'agente dipende dal progettista*: un agente è autonomo nella misura in cui il suo comportamento si basa sulla propria esperienza (percezioni e apprendimento) anziché solo sulla conoscenza inserita a priori, e opera senza un intervento umano continuo. Le due proprietà sono quindi indipendenti: un termostato con regole fisse è un agente ma non è autonomo, mentre un'auto a guida autonoma che impara e si adatta a situazioni nuove è sia intelligente sia autonoma.

1.2.6 Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali in ambienti stocastici. Gli elementi principali sono:

- **States**: insieme S degli stati; uno stato $s \in S$ rappresenta l'ambiente in un dato momento
- **Actions**: insieme A delle azioni che l'agente può intraprendere
- **Transition model**: effetto che le azioni hanno sull'ambiente. Poiché gli effetti potrebbero essere incerti, il modello assegna una probabilità a ogni possibile stato successivo

  $$\mathrm{T}(s, a, s') = P(s' \mid s, a)$$

  dove T è la funzione di transizione: non è altro che la probabilità condizionata di capitare nello stato s' partendo dallo stato s ed eseguendo l'azione a.
- **Reward**: valore **immediato** dell'esecuzione di un'azione

  $$R : S \times A \times S \to \mathbb{R}$$

  Definisce il reward che l'agente ottiene compiendo quel passo:
  - Il primo S indica lo stato di partenza
  - A è l'azione che è stata eseguita
  - Il secondo S indica lo stato in cui si è giunti
- **Policy**: strategia che l'agente utilizza per decidere quale azione intraprendere in ogni stato con l'obiettivo di massimizzare il reward totale atteso nel tempo

  $$\pi : S \to A$$

Alla base del modello c'è la **proprietà di Markov**, da cui prende il nome: la probabilità di raggiungere lo stato successivo dipende *soltanto* dallo stato corrente e dall'azione scelta, e non dalla storia degli stati e delle azioni precedenti:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

In altre parole, lo stato corrente riassume tutta l'informazione rilevante per decidere come agire.

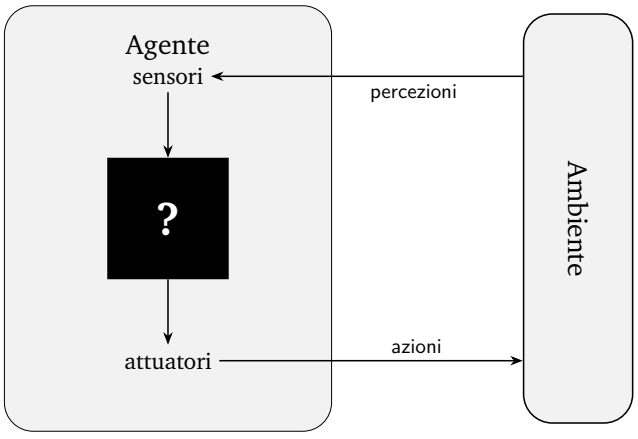
Capitolo 2

Intelligent Agents

2.1 Definizione

> **Intelligent Agent**
> Un **agente** è qualsiasi cosa possa essere vista come un sistema che percepisce il suo ambiente attraverso dei **sensori** e agisce su di esso tramite **attuatori**.

Esempio: L'agente umano possiede sensori come occhi e orecchie, mentre può usare come attuatori mani, gambe, tratto vocale. Un agente robotico potrebbe invece avere telecamere e telemetri a infrarossi come sensori e diversi motori come attuatori.



Si usa il termine **percezione** per indicare i dati che i sensori raccolgono. La **Sequenza Percettiva** di un agente è la storia di tutto ciò che l'agente

ha raccolto in tutta la sua esistenza. In generale, la scelta dell'azione di un agente in un qualsiasi istante può dipendere dalla conoscenza integrata in esso e dall'intera sequenza percettiva osservata fino a quel momento, ma non da qualcosa che l'agente non abbia percepito.

Specificando l'azione che un agente sceglie in qualunque istante, lo si definisce in modo completo. In termini matematici, il comportamento di un agente è descritto dalla **funzione agente**, che associa a ogni sequenza percettiva un'azione:

$$f : \mathcal{P}^* \to \mathcal{A}$$

In particolare:

- $\mathcal{P}$ indica l'insieme delle singole percezioni
- * (*chiusura di Kleene*) è l'operatore che, applicato a un insieme, restituisce l'insieme di *tutte* le sequenze finite (compresa quella vuota) formate dai suoi elementi
- $\mathcal{P}^*$ è quindi l'insieme di tutte le possibili sequenze percettive: ogni suo elemento rappresenta una possibile cronologia delle percezioni registrate fino a un certo momento
- $\mathcal{A}$ indica l'insieme delle possibili azioni

Internamente, la funzione agente è implementata da un **programma agente**.

Differenza – Programma Agente vs Funzione Agente

Mentre la funzione agente è una descrizione matematica astratta, il programma agente è una sua implementazione fisica concreta, in esecuzione all'interno del sistema fisico.

Esempio: Un mondo dell'aspirapolvere con due sole posizioni. Ogni posizione può essere pulita o sporca, l'agente può andare a sinistra o a destra e può pulire il riquadro in cui si trova.

Sequenza Percettiva	Azione
[*A, Pulito*]	Destra
[*A, Sporco*]	Aspira
[*B, Pulito*]	Sinistra
[*B, Sporco*]	Aspira
[*A, Pulito*], [*A, Pulito*]	Destra
[*A, Pulito*], [*A, Sporco*]	Aspira
⋮	⋮

Se un agente ha $|\mathcal{P}|$ possibili percezioni, dopo T passi temporali, la funzione agente ha $\sum_{t=1}^{T} |\mathcal{P}|^t$ voci. Se lo storico di percezioni è irrilevante, cioè se l'azione dipende soltanto dalla percezione corrente, l'agente viene detto **Simple Reflex Agent**.

L'obiettivo è progettare piccoli programmi agente per rappresentare enormi funzioni agente.

Un possibile programma agente

```
function AGENTE-REATTIVO-ASPIRAPOLVERE([posizione, stato]) returns azione
    if stato = Sporco then return Aspira
    else if posizione = A then return Destra
    else if posizione = B then return Sinistra
```

Questo programma è adeguato se la performance measure prende in considerazione solo il numero di stanze pulite. Se subentrassero altre metriche che portano a una definizione della performance measure diversa (stato della batteria, percentuale di ricarica, minimizzare il numero di spostamenti), tale programma non sarebbe in grado di esprimere la cosa migliore da fare. Si tratta di un **Simple Reflex Agent**: l'azione dipende soltanto dalla percezione corrente.

2.2 Razionalità

2.2.1 Comportarsi Correttamente: il concetto di razionalità

> **Definizione**
> Un **agente razionale** è un agente che fa la cosa giusta.

Esistono diverse accezioni dell'espressione "cosa giusta", ma l'IA rimane fedele alla nozione di **consequenzialismo**: si valuta il comportamento di un agente considerando le conseguenze. Quando un agente viene inserito in un ambiente, genera una sequenza di azioni in base alle percezioni che riceve. Questa sequenza di azioni porta l'ambiente ad attraversare una sequenza di stati: se tale sequenza è desiderabile, significa che l'agente si è comportato bene. Questa nozione di desiderabilità è catturata da una performance measure che valuta la sequenza di stati dell'ambiente.

In un dato momento, ciò che è razionale dipende da quattro fattori:

- la performance measure che definisce il criterio del successo
- la conoscenza pregressa dell'ambiente da parte dell'agente

- le azioni che l'agente può eseguire
- la sequenza percettiva dell'agente fino allo stato corrente

Da qui la definizione formale di **agente razionale**:

> **Definizione**
> Per ogni possibile sequenza di percezioni, un agente razionale dovrebbe scegliere un'azione che massimizzi il valore atteso della sua performance measure, date le informazioni fornite dalla sequenza percettiva e da ogni ulteriore conoscenza dell'agente.

La razionalità è quindi un problema di ottimizzazione sulla performance measure attesa. Essere razionale non vuol dire essere onnisciente:[1] le percezioni potrebbero non fornire tutte le informazioni rilevanti in un dato istante, e i risultati essere diversi da quelli previsti.

> **Differenza – Razionalità vs Perfezione**
> La razionalità massimizza la performance **attesa**, mentre la perfezione massimizzerebbe la performance **effettiva**. Un agente razionale non può essere giudicato per esiti che non poteva prevedere: se attraverso la strada dopo aver controllato che sia libera e vengo colpito da un oggetto caduto dall'alto, la mia scelta è stata comunque razionale.

2.2.2 Esercizio

L'esercizio propone di analizzare un ambiente composto da tre stanze (A, B, C) e due robot (r_1, r_2). Il robot r_1 può pattugliare le stanze A e B partendo da A, mentre r_2 può pattugliare B e C partendo da C. Assumendo un tempo di spostamento nullo, l'obiettivo è definire un comportamento razionale capace di minimizzare l'inattività media (*average idleness*), definita come la frazione del tempo in cui una stanza non è visitata da alcun robot.

Per affrontare il problema, si possono analizzare tre diverse strategie:

- **Prima strategia (Vettore di inattività $[0, 1, 0]$):** I robot non cambiano mai la loro posizione iniziale. Il robot r_1 rimane in A e r_2 in C. Di conseguenza, le stanze A e C hanno inattività nulla, ma la stanza B non viene mai coperta, generando un'inattività pari a 1.

[1] Possedere conoscenza illimitata su qualsiasi cosa.

- **Seconda strategia (Vettore di inattività $[1/2, 1/2, 0]$):** Si muove soltanto r_1, che si sposta continuamente tra A e B, mentre r_2 resta fermo in C: le stanze A e B rimangono scoperte per metà del tempo.
- **Terza strategia (Vettore di inattività $[1/4, 1/2, 1/4]$):** Si muove anche r_2, coordinando i due robot per coprire l'intera area: ogni robot trascorre $3/4$ del tempo nella propria stanza esterna e $1/4$ in B, e i due non si trovano mai in B contemporaneamente. La stanza B resta quindi coperta per metà del tempo.

Si noti che in tutte e tre le strategie la somma delle inattività vale 1, quindi l'**inattività media è sempre** $1/3$: rispetto all'obiettivo formale le tre strategie sono equivalenti. Tuttavia la **terza strategia è la migliore**: nella pratica conta anche coprire le stanze in modo omogeneo (*equità*), e per misurarla la media non basta: serve introdurre la **varianza** o la **deviazione standard**, che penalizzano gli squilibri estremi tra le stanze.

2.3 PEAS

Per progettare un agente intelligente bisogna definire l'ambiente in cui opera:

- **Performance Measures**: metodo di valutazione del successo dell'agente
- **Environment**: il contesto in cui l'agente opera
- **Actuators**: i mezzi attraverso cui l'agente agisce sull'ambiente
- **Sensors**: i mezzi attraverso cui l'agente percepisce l'ambiente

Esempio: si consideri un taxi automatico. Il PEAS potrebbe essere:

Performance measures	Soddisfazione del cliente, sicurezza, efficienza del carburante (SOLO E85), rispetto per le leggi stradali
Environment	Traffico stradale, condizioni meteorologiche, segnali stradali, pedoni e altri veicoli
Actuators	Volante, acceleratore, freni, indicatori di direzione
Sensors	Telecamere, GPS, sensori di velocità, radar

2.4 Tipi di Ambiente

Gli ambienti possono essere classificati secondo diverse caratteristiche:

- **Osservabile** (vs *parzialmente osservabile*): se l'agente può percepire completamente lo stato dell'ambiente in ogni momento
- **Deterministico** (vs *stocastico*): se lo stato successivo dell'ambiente è determinato completamente dallo stato corrente e dall'azione eseguita dall'agente
- **Episodico** (vs *sequenziale*): se l'esperienza dell'agente è divisa in episodi indipendenti, cioè l'azione in un episodio non influisce sugli episodi successivi
- **Statico** (vs *dinamico*): se l'ambiente non cambia mentre l'agente sta prendendo una decisione
- **Discreto** (vs *continuo*): se stati, percezioni, azioni e tempo assumono un numero finito di valori distinti
- **Singolo Agente** (vs *multi-agente*): se l'agente opera da solo nell'ambiente senza la presenza di altri agenti
- **Noto** (vs *ignoto*): se l'agente (o il progettista) conosce le regole dell'ambiente, cioè gli esiti delle azioni

Approfondimento – Scenario Singolo Agente in contesti Multi-Agente

Un ambiente multi-agente può essere considerato singolo agente in quelle circostanze in cui si prevede la progettazione di un singolo agente mentre gli altri agenti non vengono modellati (che quindi vengono considerati elementi dell'ambiente).

Esempio: Tipi di ambienti in diversi contesti

	Crossword	Robo-selector	Poker	Taxi
Observable	Yes	Partly	Partly	Partly
Deterministic	Yes	No	No	No
Episodic	No	Yes	No	No
Static	Yes	No	Yes	No
Discrete	Yes	No	Yes	No
Single-agent	Yes	Yes	No	No

Durante il corso ci si soffermerà sugli ambienti *osservabili* e *deterministici* per descrivere problemi sequenziali: un'azione svolta ha un impatto sul futuro.

2.5 Tipi di Problemi

- **Deterministico, completamente osservabile $\implies$ problema a stato singolo (Single-State Problem)**.
  L'agente sa esattamente in quale stato si troverà; la soluzione è una sequenza di azioni.
- **Non osservabile $\implies$ problema conformante**.
  L'agente potrebbe non avere idea di dove si trovi: l'agente non dispone di sensori. La soluzione (se esiste) è una sequenza di azioni.
- **Non deterministico e/o parzialmente osservabile $\implies$ problema di contingenza**.
  Le percezioni forniscono costantemente nuove informazioni sullo stato corrente; la soluzione è un piano contingente (basato su condizioni del tipo if-then) o una *policy* (mapping stato-azione): molto spesso si alternano le fasi di ricerca e di esecuzione.
- **Spazio degli stati sconosciuto $\implies$ problema di esplorazione** (*online*).
  L'agente deve scoprire gli stati e le transizioni interagendo direttamente con l'ambiente per capire come funziona il problema.

2.6 Programma Agente

Un agente è composto da un'architettura interna e da un **programma agente**, il cui scheletro è composto da:

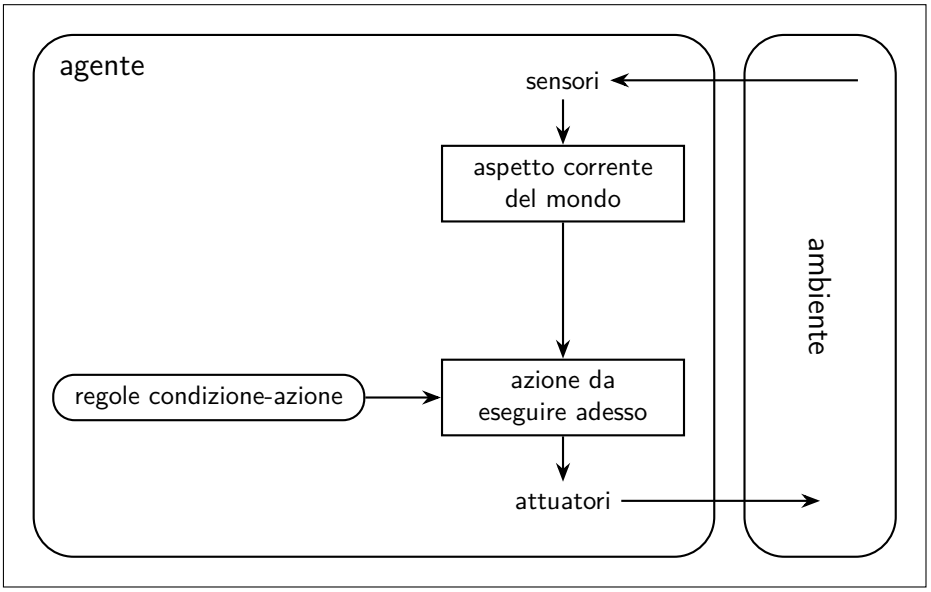
- Input: percezione **corrente**. Se l'agente necessita di una sequenza di percezioni, ogni percezione va salvata per riassemblare l'intera sequenza
- Output: **Prossima** azione

Su questo schema si costruiscono architetture diverse, con scopi diversi. I quattro tipi base, alla base di quasi tutti i sistemi intelligenti, sono:

- Simple Reflex Agents
- Reflex Agents with State
- Goal-Based Agents
- Utility-Based Agents

2.6.1 Simple Reflex Agents

L'architettura più semplice è il **Simple Reflex Agent**. Questi agenti scelgono le azioni sulla base della percezione corrente, ignorando tutta la storia percettiva precedente, seguendo **regole condizione-azione**.



Simple Reflex Agents

```
function AGENTE-REATTIVO-SEMPLICE(percezione) returns azione
    persistent: regole      // insieme di regole condizione-azione

    stato ← INTERPRETA-INPUT(percezione)
    regola ← REGOLA-CORRISPONDENTE(stato, regole)
    azione ← regola.AZIONE
    return azione
```

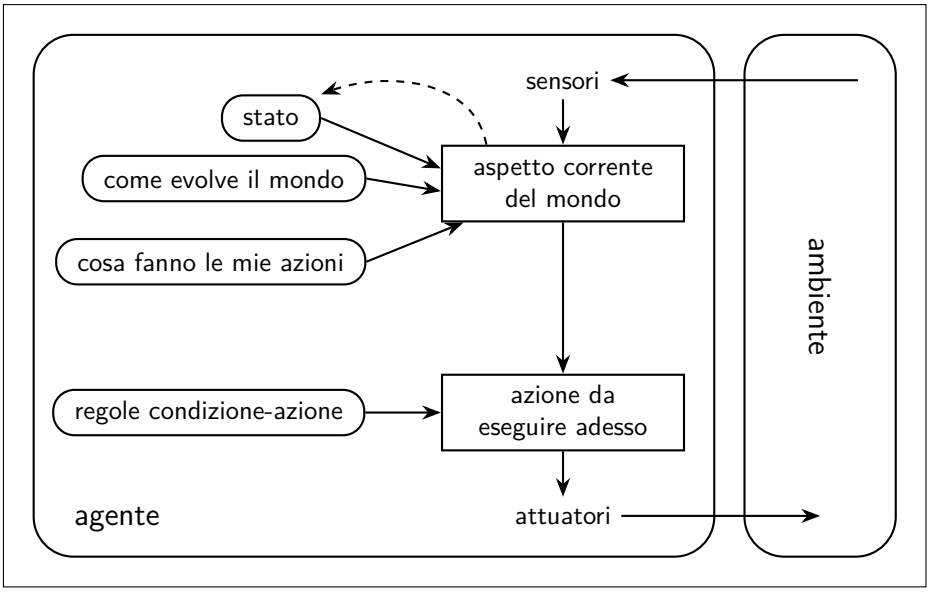
I Simple Reflex Agents hanno il pregio della semplicità, ma un'intelligenza molto limitata: funzionano correttamente **solo se l'ambiente è completamente osservabile**, cioè se la decisione giusta può essere presa sulla base della sola percezione corrente. Anche una minima parte di non osservabilità può causare seri problemi.

Esempio: se l'aspirapolvere non avesse il sensore di posizione e percepisse soltanto [*Pulito*] o [*Sporco*], davanti a una stanza pulita dovrebbe scegliere sempre la stessa azione, ad esempio *Sinistra*. Se si trova in A, quell'azione fallisce sempre: l'agente entra in un **ciclo infinito**.

2.6.2 Reflex Agents with State

Il modo più efficace di gestire l'osservabilità parziale, per un agente, è tener traccia della parte del mondo che non può vedere nell'istante corrente. Questo significa che l'agente deve mantenere una sorta di **stato interno** che dipende dalla storia delle percezioni e che quindi riflette almeno una parte degli aspetti non osservabili dello stato corrente. In particolare, aggiornare lo stato interno richiede due componenti fondamentali: gli effetti delle azioni dell'agente e le modalità di evoluzione del mondo indipendentemente dall'agente. Tale conoscenza viene implementata tramite un **transition model**. Inoltre, sono necessari **modelli sensoriali** che descrivono come lo stato del mondo si rifletta nelle percezioni dell'agente.

Un agente che utilizza tali modelli prende il nome di **Agente Basato su Modello**.



Reflex Agents with State

```
function AGENTE-REATTIVO-BASATO-SU-MODELLO(percezione) returns azione
    persistent:
        stato               // concezione corrente dello stato del mondo
        modello_transizione // stato successivo in funzione di stato e azione
        modello_sensoriale  // come lo stato si riflette nelle percezioni
        regole              // insieme di regole condizione-azione
        azione              // l'azione più recente, inizialmente nessuna

    stato ← AGGIORNA-STATO(stato, azione, percezione,
                           modello_transizione, modello_sensoriale)
    regola ← REGOLA-CORRISPONDENTE(stato, regole)
    azione ← regola.AZIONE
    return azione
```

2.6.3 Goal-Based Agents

La conoscenza dello stato corrente dell'ambiente non è sempre sufficiente per decidere quale azione intraprendere. Spesso è necessario introdurre informazioni relative all'**obiettivo** (*goal*), ovvero la descrizione delle situazioni desiderabili da raggiungere. Un *Goal-Based Agent* combina queste informazioni con il proprio modello interno del mondo: a differenza dei *Reflex Agents*, in cui l'obiettivo è solo implicito nelle regole condizione-azione preprogrammate, valuta in modo esplicito le conseguenze delle proprie scelte, verificando prima di agire se un'azione lo porterà o meno al proprio scopo.

Esempio: Il Reflex Agent frena quando vede le luci di frenata, senza rendersi conto del motivo. Un Goal-Based Agent frena quando vede le luci di frenata perché quella è l'unica azione che, secondo la sua previsione, permette di raggiungere l'obiettivo di non tamponare altre macchine.

Ricerca e **pianificazione** sono sottocampi dell'IA dedicati a identificare le sequenze di azioni da intraprendere per raggiungere gli obiettivi.

Può sembrare che un Goal-Based Agent sia meno efficiente (invece di applicare una regola già pronta, prima di agire deve simulare le conseguenze delle possibili azioni e verificare se portano all'obiettivo), ma garantisce maggiore **flessibilità**, perché la conoscenza che guida le sue decisioni è rappresentata esplicitamente e può essere modificata.

2.6.4 Utility-Based Agents

Raggiungere semplicemente un obiettivo non basta: per un risultato ottimale è fondamentale valutare anche la qualità e l'efficienza delle azioni scelte per ottenerlo (come velocità, sicurezza o convenienza).

Si introduce quindi il concetto di **Utilità**: all'interno dell'agente viene implementata una **Funzione di utilità** che internalizza la performance measure. Purché la funzione di utilità interna e la performance measure esterna concordino, un agente che sceglie le azioni per massimizzare l'utilità sarà comunque razionale in base alla performance measure esterna.

Si sottolinea che questo non è l'unico modo per essere razionali: l'esempio sull'aspirapolvere mostra la modellazione di un comportamento razionale senza introdurre obiettivi o funzioni di utilità. I vantaggi sono comunque soprattutto in termini di flessibilità, soprattutto in contesti in cui è necessario prendere decisioni in condizioni d'incertezza.

Un Utility-Based Agent razionale sceglie l'azione che massimizza l'**utilità attesa** dei risultati, cioè l'utilità media data la probabilità di ciascun risultato.

Esempio: Si immagini un taxi a guida autonoma che deve scegliere tra due strade:

- **STRADA A (Urbana)**: è sempre libera (100%) e porta a destinazione nei tempi previsti, con utilità moderata pari a 5.
  Utilità attesa: $1.0 \times 5 = 5$

- **STRADA B (Autostrada)**: è libera nell'80% dei casi, offrendo la possibilità di arrivare in largo anticipo, con utilità elevata pari a 10. Nel restante 20% dei casi, invece, si trova un ingorgo totale e si arriva molto in ritardo, con utilità $= -5$.
  Utilità attesa: $(0.8 \times 10) + (0.2 \times -5) = 8 - 1 = 7$

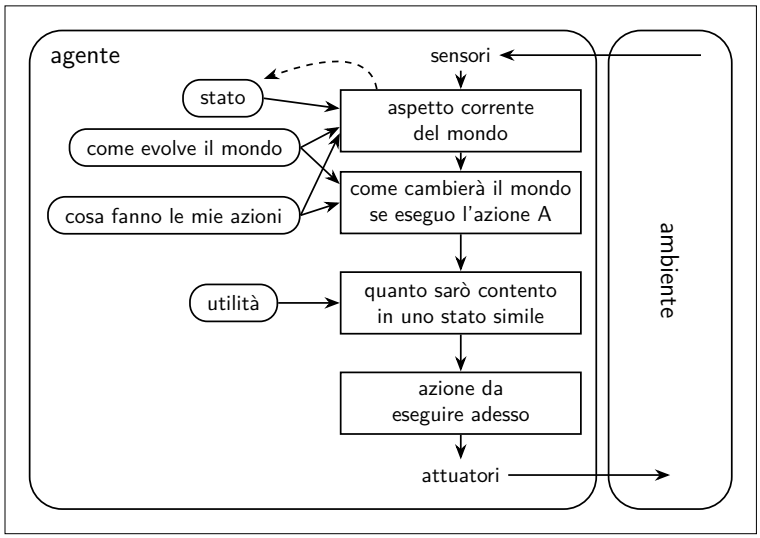
La strada che massimizza l'utilità attesa è pertanto la B.

Approfondimento – Utilità Attesa

Si noti che non è altro che il calcolo del valore medio (*speranza matematica*): $E[X] = \sum x_i \cdot P(X = x_i)$

Differenza – Funzione di Utilità vs Performance Measure

Sebbene rappresentino concetti assai simili, la Performance Measure è il metro di giudizio **esterno**, stabilito dal progettista per valutare la qualità complessiva del comportamento dell'agente: l'agente non ha necessariamente accesso diretto a essa.
La funzione di utilità è invece **interna** all'agente: è l'internalizzazione della performance measure, costruita dal progettista in modo che le due siano allineate. L'agente la usa per assegnare un valore a ogni possibile stato del mondo e, in condizioni di incertezza, sceglie l'azione che massimizza l'utilità attesa. Se la funzione di utilità non rispecchia fedelmente la performance measure, l'agente può massimizzarla e comportarsi comunque in modo non razionale rispetto al criterio esterno.



Capitolo 3

Risolvere Problemi con la Ricerca

3.1 Problem Solving Agents

Quando l'azione da svolgere non è subito evidente, un agente potrebbe avere la necessità di guardare avanti, cioè considerare una sequenza di azioni (*cammino*) che porta a uno stato obiettivo. Questo tipo di agente viene chiamato **Problem Solving Agent**. I Problem Solving Agents utilizzano rappresentazioni **atomiche** degli stati, i quali vengono considerati come entità prive di una struttura interna.

Sono una sottocategoria dei Goal-Based Agents e, per decidere quale azione intraprendere, seguono un processo in quattro macro-fasi:

1. **Formulazione dell'obiettivo (*Goal Formulation*)**: dato lo stato corrente dell'ambiente, l'agente definisce una situazione desiderabile (l'obiettivo) da raggiungere.

2. **Formulazione del problema (*Problem Formulation*)**: l'agente decide quali stati e quali azioni prendere in considerazione per raggiungere tale obiettivo.

3. **Ricerca (*Search*)**: l'agente esplora lo spazio degli stati simulando diverse sequenze di azioni nel proprio modello interno, fino a trovare una soluzione valida (una sequenza di mosse che porta all'obiettivo).

4. **Esecuzione (*Execution*)**: l'agente esegue, una alla volta, le azioni previste dalla soluzione trovata, ignorando le percezioni.

La risoluzione è quindi di tipo **offline**: la soluzione viene eseguita "a occhi chiusi", ignorando le percezioni.

Differenza – Online vs Offline Problem Solving

La tecnica offline può essere usata solo in ambienti deterministici e completamente osservabili. Nell'**Online Problem Solving**, invece, l'agente agisce con una conoscenza incompleta dell'ambiente, quindi i sensori devono restare attivi per valutare l'esito delle proprie azioni.

Un possibile programma per un Problem Solving Agent che risolve il problema mediante ricerca nello spazio degli stati:

Simple Problem-Solving Agent

```
function AGENTE-RISOLUTORE-SEMPLICE(percezione) returns azione
    persistent:
        seq         // sequenza di azioni, inizialmente vuota
        stato       // descrizione dello stato corrente del mondo
        obiettivo   // un obiettivo, inizialmente nullo
        problema    // la formulazione del problema

    stato ← AGGIORNA-STATO(stato, percezione)
    if seq è vuota then // la prima volta che si esegue
        obiettivo ← FORMULA-OBIETTIVO(stato) // che si vuole raggiungere
        problema ← FORMULA-PROBLEMA(stato, obiettivo)
        seq ← RICERCA(problema)
        if seq = failure then return null
    azione ← PRIMO(seq)
    seq ← RESTO(seq)
    return azione
```

L'agente decide cosa fare **solo** quando `seq` è vuota: formula obiettivo e problema, cerca una soluzione e la esegue ignorando le percezioni. Solo una volta esaurita la sequenza torna a formulare un nuovo obiettivo.

3.2 Come Formulare un Problema

3.2.1 Problema: Vacanze in Romania (con un M5 e39)

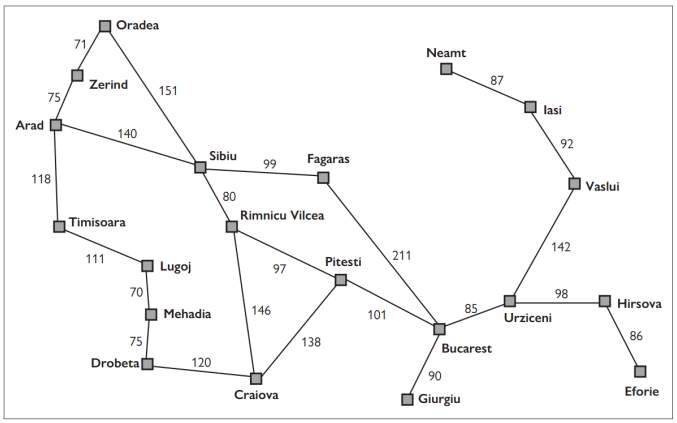
Si supponga di trovarsi in Romania, ad Arad, e di dover raggiungere Bucarest per poi prendere l'aereo e tornare a casa. L'obiettivo è trovare la serie di città da attraversare per giungere a Bucarest. Occorre individuare un corretto livello di astrazione, ma ragionevolmente si procede nel seguente modo:

- **Obiettivo**: andare a Bucarest
- **Formulazione del problema**
    1. **states**: le diverse città

2. **actions**: guidare per attraversare le città

- **Soluzione da Individuare**: sequenza di città da attraversare per arrivare a Bucarest

Il problema potrebbe non essere deterministico: a causa di agenti esterni (incidenti, lavori) si potrebbe essere dirottati su città non scelte. Per ora queste eventualità vengono ignorate. Si vuole trovare **una** soluzione: non è detto che quella trovata sia a costo minimo.



Si propone una formulazione di tale quesito come Single-State Problem, in cui si individuano i seguenti elementi:

1. **Stato Iniziale**: la città di partenza.

2. **Successor Function** $S(x)$: dato uno stato x, restituisce l'insieme delle coppie *azione–stato*, ovvero ogni azione applicabile in x insieme allo stato in cui porta. *Esempio*: $S(A) = \{\langle Arad \rightarrow Zerind, Zerind\rangle, \ldots\}$

3. **Goal Test** che può essere di due nature diverse:

   (a) esplicito: la città di destinazione (Bucarest). Lo stato in cui ci si deve trovare affinché il problema possa considerarsi risolto.

   (b) implicito: una feature che descrive più di uno stato.

4. **Path Cost**: costo di un cammino, necessario quando non basta una soluzione qualunque ma si vuole quella a costo minimo (nell'esempio, i costi espressi sugli archi). È una funzione additiva che somma gli **step cost**, funzioni del tipo STEP-COST : $S \times A \times S \rightarrow \mathbb{R}^+$.

5. **Soluzione**: sequenza di azioni che porta dallo stato iniziale a uno dei goal state.

3.3 Tree-Search Algorithm

L'idea di base è fare un'esplorazione simulata e offline: essendo un Single-State Problem, le percezioni vengono ignorate: l'agente ragiona, individua una sequenza di azioni e poi la esegue. I successori degli stati si generano usando le regole note del mondo.

Tree-Search Algorithm

```
function TREE-SEARCH(problema, strategia) returns una soluzione, o failure
    inizializza l'albero di ricerca usando lo stato iniziale del problema

    loop do
        if nessun candidato per l'espansione then return failure
        scegli uno dei nodi per l'espansione seguendo la strategia
        if il nodo contiene un goal state then return la soluzione
        else espandi il nodo e aggiungi i successori all'albero di ricerca
    end
```

3.3.1 Stati vs Nodi

Nell'albero di ricerca vanno distinti due concetti che sembrano equivalenti:

- uno **stato** è la rappresentazione di una configurazione fisica del mondo: descrive *dove* si trova l'agente (ad esempio, "sono ad Arad");
- un **nodo** è una **struttura dati** che costituisce parte dell'albero di ricerca: oltre allo stato a cui corrisponde, memorizza *come* ci si è arrivati.

I campi di un nodo sono:

State	lo stato del mondo a cui corrisponde il nodo
Parent	il nodo da cui è stato generato
Action	l'azione applicata al padre per generare il nodo
Children	i nodi generati espandendo il nodo
Depth	il numero di passi dalla radice
Path cost $g(n)$	il costo del cammino dalla radice fino al nodo

Differenza – Stato vs Nodo

Gli stati **non** hanno padre, azioni, figli, profondità o path cost: queste informazioni dipendono dal *percorso* seguito e non dal mondo. Per questo lo **stesso stato** può comparire in **più nodi diversi** dell'albero di ricerca.

Esempio: nel problema della Romania, lo stato "sono a Sibiu" può essere raggiunto:

- direttamente da Arad: profondità 1, $g = 140$;
- passando per Arad → Zerind → Oradea → Sibiu: profondità 3, $g = 75 + 71 + 151 = 297$.

Si ottengono quindi due nodi distinti, con padre, profondità e costo diversi, che contengono però lo stesso stato. In sintesi: lo stato indica *dove* si trova l'agente, il nodo indica anche *come* ci è arrivato.

3.3.2 Un'implementazione di più basso livello

Lo pseudocodice precedente è troppo astratto per essere implementato direttamente: se ne usa quindi una versione di più basso livello.

Tree-Search Algorithm

```
function TREE-SEARCH(problema, frontiera) returns una soluzione, o failure
    frontiera ← INSERISCI(CREA-NODO(problema.stato_iniziale))
    while not VUOTA(frontiera) do
        nodo ← POP(frontiera)
        if problema.GOAL_TEST(nodo.stato) then return nodo
        frontiera ← INSERISCI_TUTTI(ESPANDI(nodo, problema))
    end loop
    return failure
```

La `frontiera` è una struttura dati che contiene nodi disposti in un certo ordine. La strategia non è più un parametro in input: è determinata dal modo in cui i nodi vengono inseriti nella frontiera (Breadth First Search, Depth First Search, Best First Search …). L'algoritmo funziona così:

1. si inserisce nella frontiera il nodo corrispondente allo stato iniziale;
2. finché la frontiera non è vuota, si estrae il primo nodo (i nodi sono ordinati secondo la priorità definita dalla strategia);
3. se il nodo contiene il *goal state* viene restituito, altrimenti lo si espande secondo le regole del problema e si inseriscono i successori nella frontiera;
4. se la frontiera si svuota, si restituisce `failure`.

Lo pseudocodice della funzione `ESPANDI`:

Expand Function

```
function ESPANDI(nodo, problema) returns un insieme di nodi
    successori ← INSIEME-VUOTO()
    for each (azione, risultato) in FUNZIONE-SUCCESSORI(problema, nodo.stato) do
        s ← CREA-NODO()
        s.padre ← nodo
        s.azione ← azione
        s.stato ← risultato
        s.path_cost ← nodo.path_cost +
                                    STEP-COST(nodo.stato, azione, risultato)

        s.profondita ← nodo.profondita + 1
        AGGIUNGI(s, successori)
    end loop
    return successori
```

Nel dettaglio:

1. La funzione `FUNZIONE-SUCCESSORI` implementa la Successor Function $S(x)$: restituisce l'insieme delle coppie (`azione`, `risultato`), cioè tutte le azioni applicabili nello stato del nodo e gli stati che ne risultano. Se un'azione non altera l'ambiente, si rimane nello stesso stato di partenza.
2. Il nuovo nodo registra, oltre allo stato, il nodo padre (`padre`) e l'azione eseguita (`azione`): questa catena di puntatori permette di ricostruire l'intero percorso (la soluzione) una volta raggiunto l'obiettivo.
3. `nodo.path_cost + STEP-COST` accumula il costo del percorso dalla radice fino al nuovo nodo.
4. Tutti i nuovi nodi vengono aggiunti all'insieme `successori`, che viene restituito al chiamante.

3.3.3 Search Strategies

Una strategia è definita scegliendo l'**ordine di espansione dei nodi**. Le strategie sono valutate in base alle seguenti dimensioni:

- **completezza**: se c'è una soluzione, la ricerca è in grado di trovarla.
- **complessità temporale**: in funzione del numero di nodi generati/espansi, poiché generare e valutare un nodo richiede tempo.
- **complessità spaziale**: numero massimo di nodi in memoria (che può essere minore del numero di nodi generati).

- **ottimalità**: trova sempre la soluzione a costo minimo (quindi si desidera ottimizzare il `path_cost`, non la profondità)

Differenza – Generated vs Expanded

La **generazione** istanzia un nuovo nodo, allocando lo spazio necessario e inizializzandone gli attributi (come un costruttore nella OOP). L'**espansione** prende un nodo e, usando le regole del problema, ne genera tutti i figli da inserire nella frontiera.

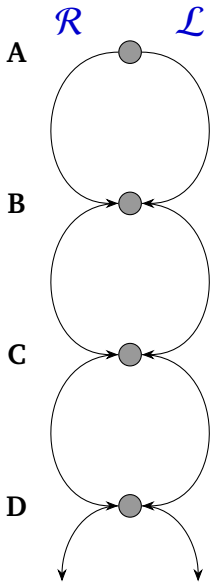
La complessità temporale e quella spaziale sono misurate in termini di:

- b: fattore di diramazione massimo dell'albero di ricerca, cioè il massimo numero di figli di un nodo.
- d: profondità della soluzione a costo minimo.
- m: profondità massima dello spazio degli stati (può essere ∞)

Si noti che m è dato dallo spazio degli stati, mentre d dalla soluzione a costo minimo (tipicamente $d \ll m$).

3.3.4 Stati Ripetuti

Dato uno stato, possono esistere diverse azioni che consentono di raggiungere lo stesso stato successivo.



Si osservi che lo stato B viene rappresentato nella frontiera **da 2 nodi distinti**: il primo generato dall'azione $\mathcal{R}$ e il secondo dall'azione $\mathcal{L}$. Questa ridondanza fa sì che la complessità del problema cresca notevolmente, assumendo un andamento di tipo esponenziale.

In generale si preferisce evitare l'espansione di stati ripetuti, ma con attenzione: esplorare i percorsi multipli può essere necessario per garantire l'ottimalità. Se la ricerca si interrompe appena viene raggiunto il goal state, si rischia di accettare una soluzione non ottimale. Nella figura, si supponga che B sia il goal state e che lo step cost di $\mathcal{R}$ sia maggiore di quello di $\mathcal{L}$: fermandosi subito dopo aver raggiunto B tramite $\mathcal{R}$, la soluzione a costo minimo (tramite $\mathcal{L}$) non verrebbe mai individuata.

Per evitare la ridondanza si tiene traccia degli stati già visitati:

Graph-Search Algorithm

```
function GRAPH-SEARCH(problema, frontiera) returns una soluzione, o failure
    esplorati ← INSIEME-VUOTO()
    frontiera ← INSERISCI(CREA-NODO(problema.stato_iniziale))
    while not VUOTA(frontiera) do
        nodo ← POP(frontiera)
        if problema.GOAL_TEST(nodo.stato) then return nodo
        if nodo.stato not in esplorati then
            AGGIUNGI(nodo.stato, esplorati)
            frontiera ← INSERISCI_TUTTI(ESPANDI(nodo, problema))
        end if
    end loop
    return failure
```

Un nodo con uno stato già esplorato non viene espanso. Resta però il rischio di perdere la soluzione ottimale: per questo sono fondamentali il momento in cui si effettua il goal test e il modo in cui si gestisce la frontiera.

3.4 Ricerca Non Informata

3.4.1 Strategie Possibili

Una **Ricerca Non Informata** utilizza solo le informazioni disponibili nella definizione del problema.

Di seguito un elenco delle strategie possibili:

- Breadth-First Search
- Depth-First Search
- Depth-Limited Search
- Uniform-Cost Search (Dijkstra)

- Iterative Deepening Search

3.4.2 Breadth-First Search

La **Breadth-First Search** (BFS, ricerca in ampiezza) espande sempre il nodo **più superficiale** tra quelli presenti nella frontiera: prima la radice, poi tutti i suoi figli, poi tutti i figli dei figli e così via. L'albero viene quindi esplorato *livello per livello*: nessun nodo a profondità $k+1$ viene espanso prima che siano stati espansi tutti i nodi a profondità k.

Per ottenere questo comportamento basta usare come frontiera una **coda FIFO**: i successori appena generati vengono inseriti in fondo alla coda, mentre il nodo da espandere viene estratto dalla testa. Poiché i nodi di un livello vengono tutti generati prima di quelli del livello successivo, la coda risulta automaticamente ordinata per profondità. L'ordine con cui vengono generati i fratelli (cioè i nodi dello stesso livello) non è invece rilevante per il funzionamento dell'algoritmo.

Breadth-First Graph Search

```
function BFS(problema) returns una soluzione, o failure
    nodo ← CREA-NODO()
    nodo.stato ← problema.stato_iniziale
    nodo.path_cost ← 0
    if problema.GOAL_TEST(nodo.stato) then return nodo
    frontiera ← CREA-FIFO()
    frontiera ← INSERISCI(nodo)
    esplorati ← INSIEME-VUOTO()
    loop do
        if VUOTA(frontiera) then return failure
        nodo ← POP(frontiera)         // il nodo più superficiale
        AGGIUNGI(nodo.stato, esplorati)
        for each azione in problema.AZIONI(nodo.stato) do
            figlio ← NODO-FIGLIO(problema, nodo, azione)
            if figlio.stato not in esplorati and figlio.stato not in frontiera then
                if problema.GOAL_TEST(figlio.stato) then return figlio
                frontiera ← INSERISCI(figlio)
            end if
        end for
    end loop
```

La funzione `NODO-FIGLIO` corrisponde a una singola iterazione del ciclo di `ESPANDI`: crea il nodo ottenuto applicando `azione` allo stato di `nodo`, impostandone padre, azione, stato, path cost e profondità.

Rispetto allo schema generale di `GRAPH-SEARCH` si notano tre differenze:

1. **La frontiera è una coda FIFO**: è l'unico elemento che caratterizza davvero la strategia.

2. **Il goal test viene anticipato**: non si controlla il nodo quando viene estratto dalla frontiera, ma *nel momento in cui viene generato* (per la radice, prima ancora di entrare nel ciclo).

3. **Il controllo sugli stati ripetuti riguarda anche la frontiera**: un figlio viene scartato non solo se il suo stato è già stato espanso, ma anche se è già presente in un nodo in attesa nella frontiera. In questo modo ogni stato compare al più in un nodo, e non si espandono mai due nodi con lo stesso stato: si tratta a tutti gli effetti di una *Graph Search*.

Approfondimento – Early Goal Test

Il controllo effettuato subito dopo la generazione prende il nome di **early goal test**. Nella BFS è lecito perché i nodi vengono generati in ordine di profondità: il primo nodo obiettivo generato è necessariamente quello alla profondità minima, quindi aspettare di estrarlo dalla frontiera non potrebbe portare a una soluzione più corta. Si risparmia però la generazione di un intero livello dell'albero (si veda la complessità temporale).
Il ragionamento **non** vale se si vuole minimizzare il *path cost* con step cost diversi (si veda la Uniform-Cost Search).

Approfondimento – Controllo degli stati nella frontiera

La frontiera contiene *nodi*, non stati: la condizione `figlio.stato not in frontiera` è un'abbreviazione tipica dello pseudocodice e significa "non esiste nella frontiera un nodo il cui attributo `stato` coincide con `figlio.stato`". Scorrere la coda a ogni controllo costerebbe tempo lineare; nelle implementazioni reali si affianca quindi alla coda un insieme (ad esempio una hash table) contenente gli stati dei nodi in frontiera, così che il controllo avvenga in tempo costante.

Proprietà

- **Completezza**: sì, se il fattore di diramazione b è finito. Se una soluzione esiste, si trova a una profondità finita d, e la BFS raggiunge ogni livello dopo aver esaurito i precedenti, che sono composti da un numero finito di nodi.

- **Complessità temporale**: nel caso pessimo il nodo obiettivo è l'ultimo a essere generato al livello d. Grazie all'early goal test l'algoritmo si ferma proprio in quel momento, dopo aver generato

$$N = 1 + b + b^2 + \cdots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

nodi. Nella notazione asintotica conta solo il termine di grado massimo, quindi la complessità temporale è $O(b^d)$.

Se invece il goal test venisse effettuato all'estrazione (dopo la `POP`), prima di arrivare al nodo obiettivo bisognerebbe espandere tutti gli altri nodi del livello d, generando fino a $b \cdot (b^d - 1) = b^{d+1} - b$ nodi del livello $d+1$. La complessità temporale salirebbe così a $O(b^{d+1})$: un intero livello in più, cioè un fattore b.

- **Complessità spaziale**: ogni nodo generato rimane in memoria, o nell'insieme `esplorati` ($O(b^{d-1})$ stati) o nella frontiera ($O(b^d)$ nodi). Lo spazio è quindi dominato dalla frontiera ed è $O(b^d)$, come il tempo. È il vero punto debole della BFS: per problemi anche di dimensione moderata la memoria si esaurisce molto prima del tempo a disposizione.
- **Ottimalità**: la BFS trova sempre la soluzione con il *minor numero di passi*, ma questa coincide con quella a costo minimo solo se tutti gli step cost sono uguali (più in generale, se il path cost è una funzione non decrescente della profondità). Con step cost arbitrari non è ottima.

3.4.3 Uniform-Cost Search

La BFS è ottima solo quando tutti gli step cost sono uguali, perché espande i nodi in ordine di *profondità*. La **Uniform-Cost Search** (UCS, ricerca a costo uniforme) generalizza l'idea: espande sempre il nodo non ancora espanso con il **path cost** $g(n)$ **minimo**, cioè il nodo più "vicino" alla radice in termini di costo invece che di numero di passi. È la versione per la ricerca dell'algoritmo di **Dijkstra** per i cammini minimi.

L'implementazione si ottiene sostituendo la coda FIFO con una **coda con priorità** ordinata per path cost crescente: la `POP` restituisce il nodo con $g(n)$ minore. Se tutti gli step cost sono uguali, ordinare per costo equivale a ordinare per profondità, e la UCS si comporta esattamente come una BFS.

Uniform-Cost Search

```
function UNIFORM-COST-SEARCH(problema) returns una soluzione, o failure
    nodo ← CREA-NODO()
    nodo.stato ← problema.stato_iniziale
    nodo.path_cost ← 0
    frontiera ← CREA-CODA-PRIORITA()  // ordinata per path_cost crescente
    frontiera ← INSERISCI(nodo)
    esplorati ← INSIEME-VUOTO()
    loop do
        if VUOTA(frontiera) then return failure
        nodo ← POP(frontiera)         // il nodo con path_cost minimo
        if problema.GOAL_TEST(nodo.stato) then return nodo
        AGGIUNGI(nodo.stato, esplorati)
```

```
    for each azione in problema.AZIONI(nodo.stato) do
        figlio ← NODO-FIGLIO(problema, nodo, azione)
        if figlio.stato not in esplorati and figlio.stato not in frontiera then
            frontiera ← INSERISCI(figlio)
        else if figlio.stato in frontiera con path_cost maggiore then
            sostituisci quel nodo della frontiera con figlio
        end if
    end for
end loop
```

Rispetto alla BFS vista prima, le modifiche necessarie per garantire l'ottimalità sono due:

1. **Il goal test si effettua quando il nodo viene selezionato per l'espansione** (subito dopo la POP), non quando viene generato.
2. **Se lo stato di un figlio è già presente nella frontiera con un path cost maggiore, il vecchio nodo viene sostituito** da quello nuovo, che rappresenta un cammino più economico verso lo stesso stato.

Approfondimento – Perché niente early goal test

Nella BFS l'early goal test funziona perché i nodi vengono *generati* in ordine di profondità: il primo nodo obiettivo generato è anche il meno profondo. Nella UCS, invece, l'ordine che conta è quello del path cost, e i nodi vengono generati in un ordine che non ha nulla a che vedere con il loro costo: quando viene generato un nodo obiettivo, nella frontiera possono esserci ancora nodi con g minore da cui parte un cammino più economico verso lo stesso obiettivo. Fermarsi alla generazione significherebbe restituire il *primo* cammino trovato, non il *migliore*.
Con il goal test all'estrazione, invece, un nodo obiettivo con costo g viene restituito solo quando è il nodo più economico dell'intera frontiera. Poiché gli step cost sono positivi, il costo lungo un cammino può solo crescere: ogni altro cammino non ancora completato costa già almeno g, e non può portare a una soluzione migliore. È esattamente il ragionamento visto nella sezione sugli stati ripetuti, con la figura delle azioni $\mathcal{R}$ e $\mathcal{L}$.

Proprietà

Sia C^* il costo della soluzione ottima e $\epsilon > 0$ il minimo step cost.

- **Completezza**: sì, se ogni step cost è $\geq \epsilon$ per qualche $\epsilon > 0$. Con step cost nulli la ricerca potrebbe bloccarsi in un ciclo infinito di azioni a

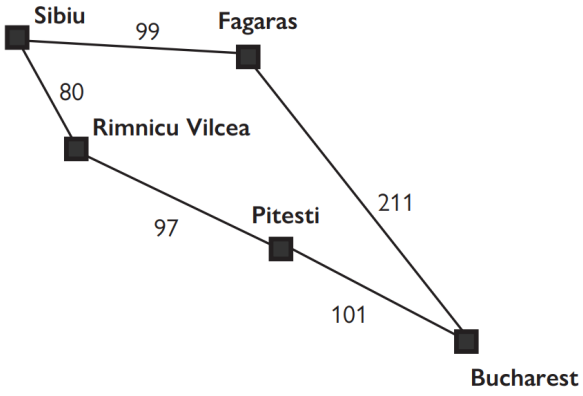
costo zero (ad esempio un'azione *NoOp* che lascia lo stato invariato), perché il path cost non crescerebbe mai.

- **Complessità temporale**: la UCS espande tutti i nodi con $g \leq C^*$. Poiché ogni passo costa almeno ϵ, un cammino di costo C^* ha al più $\lfloor C^*/\epsilon \rfloor$ passi: questo valore prende il posto della profondità d della BFS. Il +1 all'esponente deriva dal goal test all'estrazione, che costringe a generare anche i figli dei nodi dell'ultimo livello. La complessità temporale è quindi
$$O(b^{1+\lfloor C^*/\epsilon \rfloor})$$
Se tutti gli step cost valgono ϵ, si ha $C^*/\epsilon = d$ e si ottiene $O(b^{d+1})$, cioè lo stesso costo della BFS con goal test all'estrazione. Se invece esistono step cost molto piccoli, C^*/ϵ può essere molto più grande di d: la UCS può esplorare a lungo cammini fatti di tanti passi economici prima di considerare passi più costosi ma utili.

- **Complessità spaziale**: come per la BFS, i nodi generati restano in memoria, quindi anche lo spazio è $O(b^{1+\lfloor C^*/\epsilon \rfloor})$.

- **Ottimalità**: sì. I nodi vengono espansi in ordine crescente di $g(n)$, quindi il primo nodo obiettivo estratto dalla frontiera è quello con il cammino a costo minimo.

Esempio: per capire perché sono necessari questi due accorgimenti, si consideri la seguente porzione del grafo della Romania.



Si vuole arrivare a Bucarest partendo da Sibiu, lungo il percorso a costo minimo. Accanto a ogni nodo in frontiera è indicato tra parentesi il suo path cost g.

1. **Inizializzazione.** La frontiera contiene solo il nodo con lo stato iniziale:

- frontiera = {Sibiu (0)}
- esplorati = $\emptyset$

2. **Si estrae Sibiu (0).** Non è un goal state, quindi viene aggiunto agli esplorati e si espande. Si generano i nodi per i suoi successori, inizializzandone stato, padre, azione e path cost (essendo figli della radice, il path cost coincide con lo step cost):

   - Rimnicu Vilcea: step cost 80, path cost $0 + 80 = 80$
   - Fagaras: step cost 99, path cost $0 + 99 = 99$

   Stato dopo l'iterazione:

   - frontiera = {Rimnicu Vilcea (80), Fagaras (99)}
   - esplorati = {Sibiu}

3. **Si estrae Rimnicu Vilcea (80).** Non è un goal state: viene aggiunto agli esplorati e si espande.

   - Pitesti: step cost 97, path cost $80 + 97 = 177$
   - Sibiu: step cost 80, path cost $80 + 80 = 160$, ma è già esplorato, quindi il nodo viene scartato

   Stato dopo l'iterazione:

   - frontiera = {Fagaras (99), Pitesti (177)}
   - esplorati = {Sibiu, Rimnicu Vilcea}

4. **Si estrae Fagaras (99).** Non è un goal state: viene aggiunto agli esplorati e si espande.

   - Sibiu: step cost 99, path cost $99 + 99 = 198$, già esplorato, quindi scartato
   - Bucarest: step cost 211, path cost $99 + 211 = 310$

   Stato dopo l'iterazione:

   - frontiera = {Pitesti (177), Bucarest (310)}
   - esplorati = {Sibiu, Rimnicu Vilcea, Fagaras}

   Primo accorgimento. Bucarest è appena stato generato, ma il goal test non viene fatto ora. Con l'*early goal test* la ricerca si fermerebbe qui restituendo il cammino da 310, senza considerare soluzioni meno costose e perdendo l'ottimalità.

5. **Si estrae Pitesti (177).** Non è un goal state: viene aggiunto agli esplorati e si espande.

- Bucarest: step cost 101, path cost $177 + 101 = 278$
- Rimnicu Vilcea: step cost 97, path cost $177 + 97 = 274$, già esplorato, quindi scartato

Secondo accorgimento. Bucarest è già presente in frontiera, ma con un path cost più alto ($310 > 278$): il vecchio nodo viene **sostituito** da quello nuovo. Stato dopo l'iterazione:

- frontiera = {Bucarest (278)}
- esplorati = {Sibiu, Rimnicu Vilcea, Fagaras, Pitesti}

6. **Si estrae Bucarest (278).** Il suo stato è il goal state: la ricerca termina e restituisce questo nodo.

La soluzione si ricostruisce risalendo l'attributo `padre` a partire dal nodo obiettivo: Sibiu → Rimnicu Vilcea → Pitesti → Bucarest, con costo 278.

3.4.4 Depth-First Search

La DFS espande sempre il nodo **più profondo** della frontiera, che viene implementata con una politica **LIFO** (stack): i successori vengono inseriti in cima e il prossimo nodo da espandere è l'ultimo inserito.

Un aspetto importante è la **rimozione dei nodi**: quando il nodo raggiunto non contiene un goal state e non possiede successori, può essere rimosso dalla memoria. Allo stesso modo, anche un sottoalbero già visitato interamente in cui non è presente un goal state può essere rimosso. Pertanto

$$\#\text{ nodi in memoria} \leq \#\text{ nodi generati}$$

e la complessità spaziale risulta molto più bassa di quella temporale.

Proprietà

- **Completezza**: no. Fallisce se lo spazio degli stati ha profondità infinita (può scendere all'infinito lungo un ramo senza soluzioni) e in presenza di stati ripetuti (può rimanere bloccata in un ciclo).
- **Complessità temporale**: con fattore di diramazione b e profondità massima m, nel caso pessimo vengono generati

  $$1 + b + b^2 + \cdots + b^m$$

  nodi, ossia $O(b^m)$. È potenzialmente terribile, soprattutto se $m \gg d$. Se però le soluzioni sono *dense* (molti nodi contengono un goal state), la DFS può trovarne una molto prima della BFS.

- **Complessità spaziale**: $O(b \cdot m)$, ossia **lineare** nella profondità. Ogni nodo espanso genera b figli, ma se ne segue uno solo, lasciando in sospeso gli altri $b-1$. Questo accade per ciascuno dei livelli da 1 a $m-1$, mentre all'ultimo livello si trovano i b figli dell'ultimo nodo espanso. La memoria occupata è quindi proporzionale a

  $$(m-1) \cdot (b-1) + b$$

  Inoltre, quando si termina l'esplorazione di un ramo senza aver trovato un goal state, il ramo viene rimosso e si torna al nodo padre per esplorarne un altro (*backtracking*): la memoria liberata viene riutilizzata.

- **Ottimalità**: no. Restituisce la prima soluzione trovata lungo il ramo che sta esplorando, che può essere più profonda e più costosa di un'altra presente in un ramo diverso.

3.4.5 Iterative Deepening Search

È un metodo ibrido tra BFS e DFS: si eseguono ripetutamente delle DFS con un **limite di profondità** crescente. Prima si effettua una DFS con limite pari a 0 (visitando solo il nodo radice), poi con limite 1, poi 2 e così via, fino ad arrivare alla profondità in cui si trova il primo nodo con un goal state.

La tecnica è ibrida perché a ogni iterazione si visitano tutti i nodi con profondità minore o uguale al limite fissato, ottenendo un comportamento molto simile a quello della BFS (i livelli vengono raggiunti in ordine). Ogni singola ricerca, però, è una DFS e ne mantiene le caratteristiche, in particolare la complessità spaziale lineare.

Di seguito lo pseudocodice, che però assume il comportamento di `TREE-SEARCH`, consentendo l'esplorazione di nodi con stati ripetuti.

Depth-Limited Search

```
function DLS(problema, limite) returns una soluzione, o failure, o cutoff
    return R-DLS(CREA-NODO(problema.stato_iniziale), problema, limite)

function R-DLS(nodo, problema, limite) returns una soluzione, o failure, o cutoff
    if problema.GOAL_TEST(nodo.stato) then return nodo
    else if limite = 0 then return cutoff          // caso base
    else                                           // caso ricorsivo
        cutoff_avvenuto ← false
        for each azione in problema.AZIONI(nodo.stato) do
            figlio ← NODO-FIGLIO(problema, nodo, azione)
            risultato ← R-DLS(figlio, problema, limite - 1)
            if risultato = cutoff then cutoff_avvenuto ← true
            else if risultato ≠ failure then return risultato
        end for
        if cutoff_avvenuto then return cutoff
        else return failure
    end if
```

Iterative Deepening Search

```
function IDS(problema) returns una soluzione, o failure
    for profondità ← 0 to ∞ do
        risultato ← DLS(problema, profondità)
        if risultato ≠ cutoff then return risultato
    end for
```

Proprietà

- **Completezza**: sì, se il fattore di diramazione b è finito. Poiché si effettuano DLS con limite di profondità via via crescente (fino a ∞), se esiste una soluzione a profondità d la si trova all'iterazione con limite d.
- **Complessità temporale**: a ogni nuova DLS i nodi dei livelli superiori vengono rigenerati da capo, e le iterazioni sono fino a $d+1$ se la soluzione è presente (altrimenti fino a $m+1$). Il costo è quindi maggiore rispetto alla BFS, ma comunque dello stesso ordine di grandezza:

$$N_{\text{IDS}} = \underbrace{(d+1)\cdot 1}_{\text{radice}} + d\,b^1 + (d-1)\,b^2 + (d-2)\,b^3 + \cdots + 1\cdot b^d = O(b^d)$$

  I nodi a profondità i vengono generati $d+1-i$ volte, una per ogni iterazione con limite $\geq i$. In particolare il nodo con lo stato iniziale viene generato a ogni iterazione, cioè $d+1$ volte: una volta in più dei nodi a profondità 1.
- **Complessità spaziale**: $O(b \cdot d)$, ossia lineare. Ogni iterazione è una DFS con profondità massima pari al limite, quindi vale lo stesso ragionamento della DFS con d al posto di m: l'ultima iterazione, quella con limite d, è la più costosa in memoria. Si noti comunque che gli stessi nodi vengono ricreati più volte, ma poiché al termine di ogni esecuzione di DLS la memoria viene liberata, lo spazio occupato resta sempre lineare.
- **Ottimalità**: sì, se tutti gli step cost sono uguali (più in generale, se il path cost è una funzione non decrescente della profondità). Come la BFS, la IDS raggiunge i livelli in ordine e trova quindi per prima la soluzione meno profonda.

Approfondimento – Ottimalità di IDS

Solo la IDS implementata con la `TREE-SEARCH` è ottimale: se si usasse una IDS basata su `GRAPH-SEARCH`, si perderebbe l'ottimalità.

3.5 Ricerca Informata

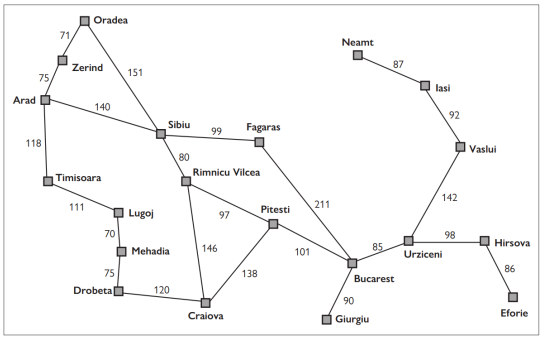
Mentre nella ricerca non informata si espande in maniera sistematica lo spazio degli stati e ciascun nodo ha lo stesso valore, nella **Ricerca Informata** si utilizzano informazioni esterne per capire se ci sono nodi che potrebbero essere migliori di altri.

Per affrontare gli algoritmi di Ricerca Informata è prima necessario introdurre le **Euristiche**: indicazioni fornite all'algoritmo per indirizzare la ricerca in determinate direzioni piuttosto che in altre.

L'idea alla base della Ricerca Informata consiste nell'utilizzare una *funzione di valutazione* che stima, per ogni nodo, la desiderabilità del nodo stesso. La funzione di valutazione utilizza le euristiche per espandere il nodo più desiderabile non ancora esplorato. Questa tecnica si chiama **best first search**. È importante notare che la frontiera è ordinata per desiderabilità decrescente. A seconda di come viene definita la desiderabilità, si distinguono due tipologie di ricerca informata:

- **Greedy Best-First Search**
- $\mathcal{A}^*$ **Search**

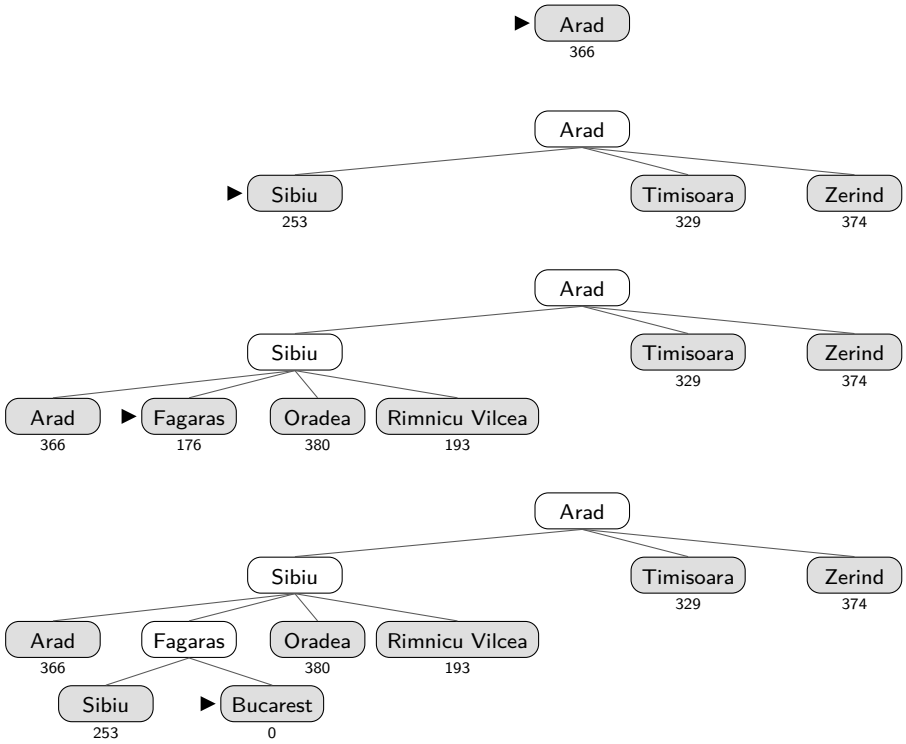
Per comprendere al meglio il funzionamento di questi algoritmi di ricerca, si consideri nuovamente il grafo topologico della Romania, con un nuovo parametro: h_{DLA}, la distanza in linea d'aria di ogni città da Bucarest.



Città	h_{DLA}	Città	h_{DLA}
Arad	366	Mehadia	241
Bucarest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

3.5.1 Greedy Search

La funzione di valutazione $h(n)$ consente di stimare il costo del nodo n dal più vicino nodo con goal state. *Esempio*: $h_{\text{SLD}}(n)$: distanza in linea d'aria da n a Bucarest. L'algoritmo è greedy perché a ogni iterazione si espande il nodo che sembra essere **più vicino** al nodo con stato di goal, secondo le euristiche fornite.



Dall'alto: stato iniziale e albero dopo l'espansione di Arad, Sibiu e Fagaras. In grigio i nodi della frontiera, con sotto il valore di h_{SLD}; il triangolo indica il prossimo nodo da espandere.

Proprietà

- **Completezza**: No, può bloccarsi in Loop
  Esempio: Start: Iasi, Goal: Fagaras
  Iasi → Neamt → Iasi → …
- **Complessità temporale**: $O(b^m)$ se l'euristica non funziona correttamente potrebbe essere necessario esplorare continuamente in profondità per trovare il nodo con stato di goal, spingendo ogni volta sul ramo errato fino alla massima profondità m.
- **Complessità spaziale**: $O(b^m)$
- **Ottimalità**: No, basta guardare il caso descritto nell'esempio della Romania.

Sebbene l'utilizzo di un'euristica molto ottimizzata consenta di limitare le criticità di Best First Search, una soluzione migliore in alcuni contesti corrisponde all'utilizzo di $\mathcal{A}^*$.

L'idea di base consiste nell'evitare l'espansione di tutti quei nodi in percorsi costosi. La funzione di valutazione prende la seguente forma:

$$f(n) = g(n) + h(n)$$

dove, dato un nodo n:

- $g(n)$: somma degli step cost per raggiungere n (quindi il path cost per arrivare a n)
- $h(n)$: euristica per raggiungere il nodo con goal state da n.
- $f(n)$: costo stimato per raggiungere il nodo con goal state passando per n

I nodi vengono ordinati in frontiera in modo crescente rispetto al valore $f(n)$.

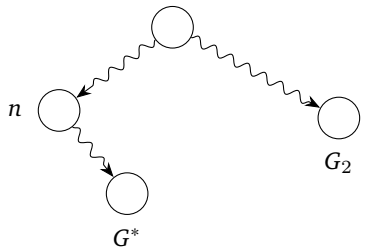
La precondizione per consentire l'uso di $\mathcal{A}^*$ prevede l'uso di euristiche ammissibili:

$$h(n) \leq h^*(n) \text{ dove } h^*(n) \text{ è il costo reale da } n.$$

Si richiede inoltre che $h(n) \geq 0$. Insieme all'ammissibilità, questo implica che $h(G) = 0$ per ogni nodo con stato di goal G: infatti $0 \leq h(G) \leq h^*(G) = 0$.

Teorema 3.5.1 (Ottimalità di $\mathcal{A}^*$)**.** Se l'euristica $h(n)$ è ammissibile e $\mathcal{A}^*$ è implementato con `TREE-SEARCH`, allora $\mathcal{A}^*$ è ottimale.

Dimostrazione. Si supponga che nella frontiera si trovi un nodo obiettivo *sub-ottimo* G_2. Sia G^* il nodo obiettivo ottimo e sia n un nodo della frontiera non ancora espanso che si trova sul cammino ottimo verso G^*.



Si vuole mostrare che $f(G_2) > f(n)$, cioè che $\mathcal{A}^*$ espande n (e quindi prosegue verso G^*) prima di selezionare G_2.

1. Poiché G_2 è un goal state, $h(G_2) = 0$, quindi

$$f(G_2) = g(G_2) + h(G_2) = g(G_2)$$

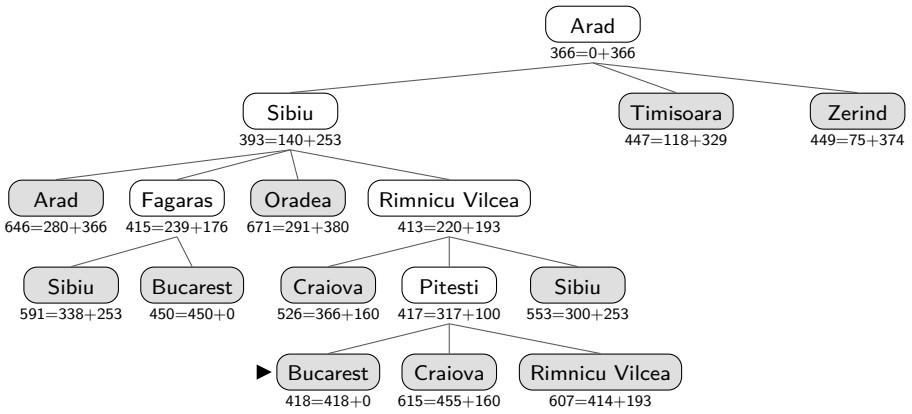
2. Poiché G_2 è sub-ottimo, il suo costo è maggiore di quello della soluzione ottima:
$$g(G_2) > g(G^*)$$

3. Il costo di G^* è il costo per arrivare a n più il costo reale da n a G^*. Poiché l'euristica è ammissibile, $h(n) \leq h^*(n)$, quindi
$$g(G^*) = g(n) + h^*(n) \geq g(n) + h(n) = f(n)$$

Mettendo insieme i tre passaggi:
$$f(G_2) = g(G_2) > g(G^*) \geq f(n) \quad \Longrightarrow \quad f(G_2) > f(n)$$

Quindi n viene sempre espanso prima di G_2. Lo stesso ragionamento vale per ogni nodo del cammino ottimo (formalmente si dimostra per **induzione** sui nodi del cammino): $\mathcal{A}^*$ raggiunge G^* prima di selezionare G_2. □

Esempio: $\mathcal{A}^*$ sul problema della Romania, da Arad a Bucarest. Essendo una `TREE-SEARCH`, alcuni stati compaiono in più nodi (ad esempio Arad e Sibiu). Sotto ogni nodo è indicato $f(n) = g(n) + h(n)$.



Albero di ricerca dopo l'espansione di Pitesti. In bianco i nodi espansi (nell'ordine: Arad, Sibiu, Rimnicu Vilcea, Fagaras, Pitesti), in grigio quelli della frontiera; il triangolo indica il prossimo nodo da espandere.

- **Il comportamento di $\mathcal{A}^*$ è diverso sia da quello della BFS sia da quello della DFS**: lo si nota dopo l'espansione di Rimnicu Vilcea. Il nodo della frontiera con $f(n)$ più basso è Fagaras ($415 < 417$ di Pitesti) , quindi la ricerca riprende da lì, su un altro ramo, invece di scendere sotto Rimnicu Vilcea come farebbe la DFS. Allo stesso tempo non procede per livelli come la BFS, che avrebbe espanso Timisoara e Zerind (profondità 1) prima dei nodi a profondità 2.

- Espandendo Fagaras si genera Bucarest con $f = 450$: è un goal state *sub-ottimo*, il G_2 della dimostrazione. Poiché il goal test si effettua all'estrazione, la ricerca non si ferma: in frontiera ci sono nodi con costo più basso, come Pitesti ($417 < 450$), che vengono espansi prima.
- Solo espandendo Pitesti si ottiene il Bucarest ottimo G^*, con $f = 418$. Poiché $f(G^*) = 418 < f(G_2) = 450$, G^* viene posizionato prima di G_2 nella frontiera ed estratto per primo. La soluzione è Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucarest, con costo 418.

Proprietà

- **Completezza**: Si, a patto che non vi sia un numero infinito di nodi con $f < f(G)$. Ma questo implica che lo spazio degli stati è infinito.
- **Complessità temporale**: Esponenziale in

  [errore relativo dell'euristica h × profondità della soluzione d]

  La complessità dipenda da quanto h sbaglia rispetto al costo vero. Pertanto, a differenza di Best First Search, si ha una garanzia che più è precisa, minore saranno entrambe le complessità.
- **Complessità spaziale**: Come la complessità temporale.
- **Ottimalità**: Si, a patto che l'euristica sia ammissibile nel caso di `TREE-SEARCH`, altrimenti se si propone un'implementazione `GRAPH-SEARCH`, allora serve una precondizione più potente: la **consistenza**.

3.5.2 Consistenza

Con `GRAPH-SEARCH` l'ammissibilità non basta più a garantire l'ottimalità. Un nodo il cui stato è già in `esplorati` viene scartato: se $\mathcal{A}^*$ espande uno stato la prima volta lungo un cammino sub-ottimo, il cammino migliore trovato in seguito viene perso. Serve quindi una condizione più forte, che garantisca che **la prima volta che uno stato viene espanso, lo si è raggiunto con il cammino ottimo**.

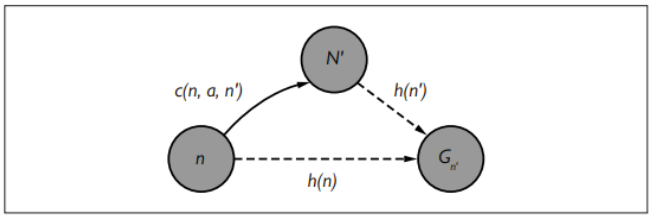
Euristica consistente

Un'euristica h è **consistente** (o **monotona**) se, per ogni nodo n e per ogni successore n' generato da un'azione a,

$$h(n) \leq c(n, a, n') + h(n')$$

e $h(G) = 0$ per ogni nodo con stato di goal G.

La condizione è una forma di **disuguaglianza triangolare**: nel triangolo formato da n, n' e dal goal $G_{n'}$, il lato $h(n)$ non può superare la somma degli altri due, $c(n, a, n')$ e $h(n')$.



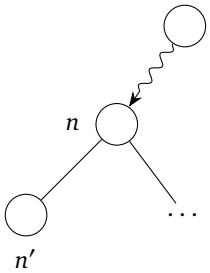
Spostando $h(n')$ a sinistra si ottiene una lettura equivalente:

$$h(n) - h(n') \leq c(n, a, n')$$

cioè, passando da n a n', la stima del costo rimanente può diminuire al più di quanto si è effettivamente pagato per quel passo. Se diminuisse di più, le due stime sarebbero incoerenti tra loro: da n si può raggiungere il goal passando per n', con un costo stimato di $c(n, a, n') + h(n')$, quindi non ha senso stimare $h(n)$ più alto.

Consistenza e ottimalità di $\mathcal{A}^*$

Si consideri un nodo n e un suo successore n'.



Si vuole che f non diminuisca passando da n a n'. Poiché n' è un successore di n (tramite l'azione a), vale $g(n') = g(n) + c(n, a, n')$, quindi:

$$\begin{aligned} f(n) &\leq f(n') \\ g(n) + h(n) &\leq g(n') + h(n') \\ g(n) + h(n) &\leq g(n) + c(n, a, n') + h(n') && \text{sostituendo } g(n') \\ h(n) &\leq c(n, a, n') + h(n') && \text{semplificando } g(n) \end{aligned}$$

L'ultima riga è proprio la definizione di consistenza.

Ne segue che $\mathcal{A}^*$ espande i nodi in ordine di f **non decrescente**: quando si espande n, i nodi in frontiera hanno $f \geq f(n)$ (n era il minimo) e, per quanto visto sopra, anche i figli di n hanno $f \geq f(n)$. Non può quindi mai capitare di estrarre un nodo con f più basso di uno già espanso.

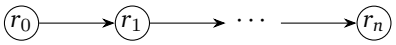
Differenza – Ammissibilità vs Consistenza

In generale essere consistenti è una condizione sufficiente ma non necessaria per essere ammissibili; viceversa, essere ammissibili non vuol dire essere consistenti:

$$\begin{aligned} \text{Consistenza} &\implies \text{Ammissibilità} \\ \text{Ammissibilità} &\not\implies \text{Consistenza} \end{aligned}$$

Esempio: euristica ammissibile ma non consistente

Si consideri una catena di stati da r_0 (stato iniziale) a r_n (goal), in cui ogni passo costa 1:



Indicando con $c(i)$ il costo reale da r_i al goal e con $h(i)$ l'euristica in r_i:

$$c(i) = n - i \qquad h(i) = n - 2\left\lceil \frac{i}{2} \right\rceil$$

Ammissibilità. Bisogna verificare che $h(i) \leq h^*(i)$ per ogni i, dove $h^*(i) = c(i)$:

$$n - 2\left\lceil \frac{i}{2} \right\rceil \leq n - i \quad \iff \quad 2\left\lceil \frac{i}{2} \right\rceil \geq i$$

che è sempre vera: per i dispari vale $>$, per i pari vale $=$. Quindi h è ammissibile.

Consistenza. Per mostrare che h non è consistente basta trovare un i per cui

$$h(i) > c(i, i+1) + h(i+1)$$

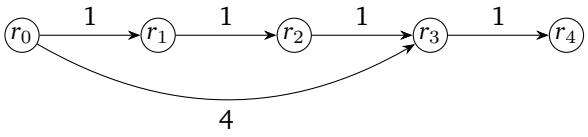
Con $i = 0$, poiché $c(0,1) = 1$ e $\lceil 0.5 \rceil = 1$:

$$\begin{aligned} n - 2\left\lceil \frac{0}{2} \right\rceil &> 1 + n - 2\left\lceil \frac{1}{2} \right\rceil \\ n &> 1 + n - 2 = n - 1 \end{aligned}$$

La disuguaglianza è vera, quindi h non è consistente.

Esempio: euristica non consistente e `GRAPH-SEARCH`

Con un'euristica non consistente, $\mathcal{A}^*$ con `GRAPH-SEARCH` può restituire una soluzione sub-ottima. Costo ed euristica sono definiti come nell'esempio precedente, con $n = 4$; in più si aggiunge un'azione che porta direttamente da r_0 a r_3 con costo 4.



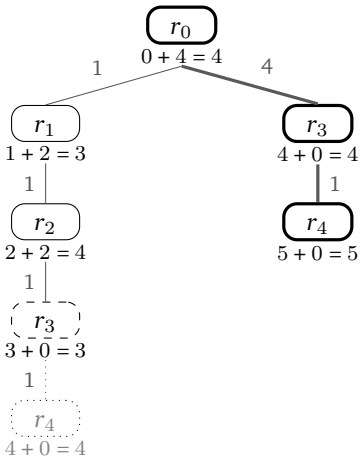
h	4	2	2	0	0
$h^* = c$	4	3	2	1	0

L'euristica non è consistente, infatti:

$$h(0) = 4 \ > \ c(0,1) + h(1) = 1 + 2 = 3$$

cioè passando da r_0 a r_1 l'euristica diminuisce di $h(0) - h(1) = 2$, più del costo del passo.

Si esegua $\mathcal{A}^*$ con `GRAPH-SEARCH`; sotto ogni nodo è indicato $g + h = f$.



Nodi espansi nell'ordine: r_0, r_1, r_3, r_2. Tratteggiato il nodo scartato perché il suo stato è già in `esplorati`, punteggiato il nodo mai generato, in grassetto il cammino restituito.

1. Si espandono r_0 e poi r_1 ($f = 3$). In frontiera restano r_3 e r_2, entrambi con $f = 4$: a parità di f si estrae r_3, inserito per primo, che genera r_4 con $f = 5$. Ora `esplorati` $= \{r_0, r_1, r_3\}$.
2. Si espande r_2, che genera r_3 con $f = 3$: è il nodo con f più basso, ma quando viene estratto il suo stato è già in `esplorati`, quindi viene scartato. Il cammino ottimo $r_0 \to r_1 \to r_2 \to r_3 \to r_4$, di costo 4, non viene mai completato.

3. Si estrae r_4 con $f = 5$: la soluzione restituita è $r_0 \to r_3 \to r_4$, di costo 5, sub-ottima.

Riepilogo

$$\texttt{TREE-SEARCH} + \text{euristica ammissibile} \implies \text{ottimalità di } \mathcal{A}^*$$
$$\texttt{GRAPH-SEARCH} + \text{euristica ammissibile} \not\implies \text{ottimalità di } \mathcal{A}^*$$
$$\texttt{GRAPH-SEARCH} + \text{euristica consistente} \implies \text{ottimalità di } \mathcal{A}^*$$

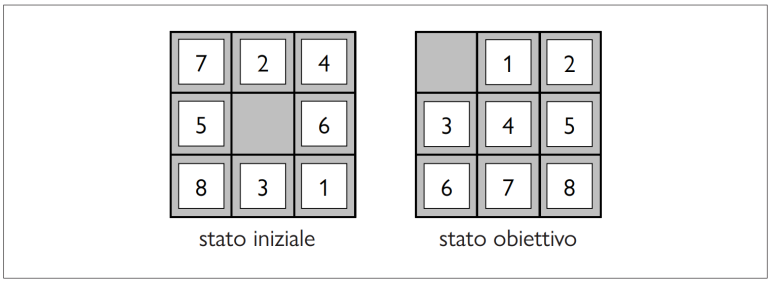
Esempio: euristiche per l'8-puzzle

Per l'8-puzzle si possono definire, ad esempio, le seguenti euristiche:

- $h_1(n) =$ numero di tessere fuori posto;
- $h_2(n) =$ distanza di Manhattan totale, cioè la somma, per ogni tessera, del numero di caselle che la separano dalla posizione desiderata.

La **distanza di Manhattan** è una metrica che rappresenta la distanza tra due punti come somma dei valori assoluti delle differenze delle loro coordinate:

$$d\big((x_1, y_1), (x_2, y_2)\big) = |x_1 - x_2| + |y_1 - y_2|$$



Per lo stato iniziale S in figura:

- $h_1(S) = 8$: tutte le tessere sono fuori posto;
- $h_2(S) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$, dove ogni termine è la distanza di Manhattan di una tessera, dalla 1 alla 8.

3.5.3 Dominanza

Dominanza

Siano h_1 e h_2 due euristiche ammissibili. Se $h_2(n) \geq h_1(n)$ per ogni nodo n, allora h_2 **domina** h_1 ed è migliore per la ricerca.

Date due euristiche ammissibili qualsiasi h_a e h_b, l'euristica

$$h(n) = \max\left(h_a(n), h_b(n)\right)$$

è anch'essa ammissibile e domina sia h_a sia h_b.

3.5.4 Problemi Rilassati

Si possono ricavare euristiche ammissibili dal costo esatto della soluzione di una **versione rilassata** del problema, ovvero un problema da cui sono stati eliminati alcuni vincoli, rendendolo più semplice da risolvere. Ad esempio, per l'8-puzzle:

- se le regole vengono rilassate in modo che una tessera possa essere spostata **ovunque**, allora $h_1(n)$ dà il costo della soluzione più breve;
- se le regole vengono rilassate in modo che una tessera possa essere spostata in una **qualsiasi casella adiacente**, allora $h_2(n)$ dà il costo della soluzione più breve.

Capitolo 4

Esercizi

Esercizio 1

> Un uomo ha un lupo, una pecora e un cavolo. L'uomo è sulla riva di un fiume con una barca che può trasportare solo lui e un altro oggetto. Il lupo mangia la pecora e la pecora mangia il cavolo, quindi non può lasciarli insieme da soli.

Prima di risolvere il problema è necessario formalizzarlo: in particolare, lo si vuole formalizzare come un problema di ricerca e poi risolvere. La soluzione proposta prevede la formalizzazione degli stati con una tupla usata per mappare le posizioni del lupo, della pecora, del cavolo, dell'uomo e della barca:

$$\langle W, S, C, M, B \rangle$$

Ciascuna delle variabili ha come dominio l'insieme $\{0, 1\}$, i cui valori corrispondono all'argine destro e a quello sinistro del fiume. In particolare:

- W: posizione del lupo
- S: posizione della pecora
- C: posizione del cavolo
- M: posizione dell'uomo
- B: stato della barca

Lo stato iniziale del problema corrisponde alla tupla

$$\langle 0, 0, 0, 0, 0 \rangle$$

mentre il goal state è

$$\langle 1, 1, 1, 1, 1 \rangle$$

Adesso ci si occupi delle azioni, che possono essere definite nel seguente modo:

- Porta il lupo (CW)
- Porta la pecora (CS)
- Porta il cavolo (CC)
- Non porta niente (CN)

Non tutte le azioni possono essere eseguite in ogni stato. La seguente tabella esprime le condizioni affinché un'azione possa essere eseguita e il risultato della sua applicazione.

Operatore	Precondizione	Risultato
CW	$M = B, M = W, S \neq C$	$\langle W, S, C, M, B\rangle \mapsto \langle \bar{W}, S, C, \bar{M}, \bar{B}\rangle$
CS	$M = B, M = S$	$\langle W, S, C, M, B\rangle \mapsto \langle W, \bar{S}, C, \bar{M}, \bar{B}\rangle$
CC	$M = B, M = C, W \neq S$	$\langle W, S, C, M, B\rangle \mapsto \langle W, S, \bar{C}, \bar{M}, \bar{B}\rangle$
CN	$M = B$	$\langle W, S, C, M, B\rangle \mapsto \langle W, S, C, \bar{M}, \bar{B}\rangle$

Si noti che M (uomo) e B (barca) assumono sempre lo stesso valore: la condizione $M = B$ è infatti sempre presente. Poiché descrivono la stessa informazione, si potrebbero fondere le due variabili in una sola (M).